UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Hex Fit Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Hex Fit

Hex Fit is a standalone editable Unity game template. Drag a comb from the tray onto empty honeycomb cells. Fill complete lines along the hex board axes to clear them. Meet the level goal before the comb budget runs out. Some boards contain flowers, bees, wax or holes; inspect the displayed goal. Meadow endless mode unlocks at level 5 and has separate high-score play.

Includes 25 curated levels, level generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Hex Fit > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/HexFit/Scenes/HexFit_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/HexFit`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/hexfit contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/HexFitGame.cs`
- `Scripts/HexFitGame.Hud.cs`
- `Scripts/HexFitGame.View.cs`
- `Scripts/HfArt.cs`
- `Scripts/HfData.cs`
- `Scripts/HfGen.cs`
- `Scripts/HfLevels.cs`
- `Scripts/HfLogic.cs`
- `Scripts/HfSession.cs`
- `Scripts/HfSfx.cs`

Main flow: HexFitGame.cs. UI and popup consumers: HexFitGame.Hud.cs. Model/generator/loading: HfData.cs, HfGen.cs, HfLogic.cs, HfSession.cs, HfLevels.cs. Authoring: Editor/HexFitLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag a comb from the tray onto empty honeycomb cells. Fill complete lines along the hex board axes to clear them. Meet the level goal before the comb budget runs out. Some boards contain flowers, bees, wax or holes; inspect the displayed goal. Meadow endless mode unlocks at level 5 and has separate high-score play.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. BoostName, BoostShort, BoostUnlock and BoostStart in HexFitGame.cs; BoostDesc and guide strings in HexFitGame.Hud.cs.

Helper	Unlock	Free charges	Action and effect
Spin	3	3	Arm Spin and tap a nonsymmetric multi-cell tray comb to rotate it 60 degrees.
Smoker	6	2	Arm Smoker and tap the hive to clear that cell and its six neighbors; eligible flowers/bees count.
Fresh Combs	10	2	Deals the next fresh tray pieces.
Drip	15	2	Arm Drip and tap a multi-cell tray comb to turn it into a single drop.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. Undo needs an eligible previous move, Scoop needs sand, Spin needs a nonsymmetric comb and Drip needs a multi-cell comb. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Hex Fit > Level Editor**. Exact implementation: `Editor/HexFitLevelEditor.cs`.

Duplicate an early board. Set Name, First-time hint, Goal and target, Comb budget, Breather, Deal difficulty and Seed. Opening combs stores explicit shape/turn codes; Clear queue removes that fixed opening. Hive radius and shape affect board geometry; Apply shape replaces the layout. Left-drag paints, right-click empties a cell. Set flowers/bees/wax/holes deliberately. Calibrate measures a suitable target/budget with the bot. Validate and Save before Play this level.

1. Back up Resources/hexfit/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`HexFitLevelEditor.cs:45`	`[MenuItem("Szekipapa77/Hex Fit/Level Editor")]`
`HexFitLevelEditor.cs:48`	`[MenuItem("Szekipapa77/Hex Fit/Validate All Levels")]`
`HexFitLevelEditor.cs:109`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`HexFitLevelEditor.cs:110`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`HexFitLevelEditor.cs:113`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`HexFitLevelEditor.cs:114`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

HexFitLevelEditor.cs:117	if (GUILayout.Button(new GUIContent("Play this level", "Optional quick test: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(100)) && Cur != null) Play(_sel + 1);
HexFitLevelEditor.cs:145	if (GUILayout.Button("Add"))
HexFitLevelEditor.cs:151	if (GUILayout.Button("Dup") && Cur != null) { var c = Cur.Clone(); c.name += " copy"; Insert(_sel + 1, c); }
HexFitLevelEditor.cs:152	if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))
HexFitLevelEditor.cs:156	if (GUILayout.Button("Up") && _sel > 0) Move(-1);
HexFitLevelEditor.cs:157	if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Move(1);
HexFitLevelEditor.cs:187	EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.boldLabel);
HexFitLevelEditor.cs:188	l.name = EditorGUILayout.TextField("Name", l.name);
HexFitLevelEditor.cs:189	l.hint = EditorGUILayout.TextField(new GUIContent("First-time hint", "Shown on the level card (introduce new mechanics here)"), l.hint);
HexFitLevelEditor.cs:192	l.goal = GoalNames[EditorGUILayout.Popup("Goal", gi, GoalNames, GUILayout.Width(300))];
HexFitLevelEditor.cs:193	if (l.Goal == GoalKind.Lines || l.Goal == GoalKind.Score) l.target = EditorGUILayout.IntField(l.Goal == GoalKind.Score ? "Target points" : "Target lines", l.target, GUILayout.Width(260));
HexFitLevelEditor.cs:194	else EditorGUILayout.LabelField($"Target: all {l.GoalTarget} {(l.Goal == GoalKind.Flowers ? "flowers" : "bees")} on the board", GUILayout.Width(300));

`HexFitLevelEditor.cs:197`	`l.budget = EditorGUILayout.IntSlider(new GUIContent("Comb budget", "Combs the player may place"), l.budget, 1, 120);`
`HexFitLevelEditor.cs:198`	`l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(80));`
`HexFitLevelEditor.cs:201`	`l.difficulty = EditorGUILayout.Slider(new GUIContent("Deal difficulty", "Bigger, awkward combs as it rises"), l.difficulty, 0, 1);`
`HexFitLevelEditor.cs:202`	`l.seed = EditorGUILayout.IntField(new GUIContent("Seed", "Comb colours and the fixed comb sequence"), l.seed, GUILayout.Width(240));`
`HexFitLevelEditor.cs:205`	`if (GUILayout.Button(new GUIContent("Calibrate", "Bot-measures the budget (flowers/bees) or the target (lines/score)"), GUILayout.Width(80))) Calibrate(l);`
`HexFitLevelEditor.cs:206`	`if (GUILayout.Button(new GUIContent("New seed", "Re-rolls the comb sequence and colours"), GUILayout.Width(80))) { l.seed = (Environment.TickCount & 0xffff) + 1; Changed(); }`
`HexFitLevelEditor.cs:207`	`if (GUILayout.Button("Clear queue", GUILayout.Width(85))) { l.queue = ""; Changed(); }`
`HexFitLevelEditor.cs:209`	`l.queue = EditorGUILayout.TextField(new GUIContent("Opening combs", "Fixed comb codes dealt first (e.g. 2.0 3I.1 4O.0); after them the seeded sequence continues"), l.queue);`
`HexFitLevelEditor.cs:211`	`EditorGUILayout.LabelField("Comb codes: " + string.Join(" ", Shapes.Bases.Select(b => b + ".0")) + " (the number after the dot is the turn)", EditorStyles.miniLabel);`
`HexFitLevelEditor.cs:214`	`int rad = EditorGUILayout.IntSlider(new GUIContent("Hive radius", "Cells from the centre to an edge (side = radius + 1)"), l.radius, 2, 6);`
`HexFitLevelEditor.cs:217`	`_shape = HfGen.ShapeNames[EditorGUILayout.Popup("Hive shape", si, HfGen.ShapeNames, GUILayout.Width(300))];`

HexFitLevelEditor.cs:218	if (GUILayout.Button("Apply shape", GUILayout.Width(90))) { ApplyShape(l, _shape); Changed(); }
HexFitLevelEditor.cs:219	if (GUILayout.Button("Clear hive", GUILayout.Width(80))) { foreach (var (q, r) in Cells(l)) if (l.At(q, r) != '#') l.Set(q, r, '.'); Changed(); }
HexFitLevelEditor.cs:227	EditorGUILayout.LabelField("Left-click/drag paints the brush. Right-click makes the cell empty.", EditorStyles.miniLabel);
HexFitLevelEditor.cs:229	EditorGUILayout.LabelField($"Cells {CountReal(l)} Combs {l.Count('p', 'b', 'w')} Bees {l.Bees} Wax {l.Wax} Flowers {l.Flowers} Holes {l.Voids} Lines {l.BuildBoard().Lines.Length}", EditorStyles.miniBoldLabel);
HexFitLevelEditor.cs:231	EditorGUILayout.HelpBox("Solver: " + v + (v.Solvable ? (v.WeakRate < 0.2f ? "\nVery hard for casual players (casual bots rarely make it)." : "") : "\nNOT solvable - raise the budget, lower the target or remove obstacles."),
HexFitLevelEditor.cs:234	EditorGUILayout.HelpBox(string.Format(CultureInfo.InvariantCulture, "Last saved validation: {0}, bots {1:0}%, best line {2:0} combs, difficulty {3:0}. Press Validate to re-run.", l.solved ? "solvable" : "NOT solvable", l.botRate * 100, l.botPieces, l.score), l.solved ? MessageType.None : MessageType.Error);
HexFitLevelEditor.cs:346	EditorGUILayout.LabelField("Difficulty curve (validated score, else last saved score / deal difficulty; blue = breather). Click a bar to select.", EditorStyles.miniBoldLabel);
HexFitLevelEditor.cs:368	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, solver-validated, same HfGen code as the runtime)", true);
HexFitLevelEditor.cs:371	_genCount = Mathf.Clamp(EditorGUILayout.IntField("Count", _genCount), 1, 100); _genStart = Mathf.Max(1, EditorGUILayout.IntField("Start at level #", _genStart));

`HexFitLevelEditor.cs:372`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
`HexFitLevelEditor.cs:375`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`HexFitLevelEditor.cs:378`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8);`
`HexFitLevelEditor.cs:379`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`HexFitLevelEditor.cs:390`	`if (GUILayout.Button($"Generate {_genCount} level(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Count 1–100; Start at level #; Seed; Start/End difficulty; Breather every 0–8; Replace that range (else insert). Checked replaces existing records in the range; unchecked inserts and shifts subsequent records. Start 26 appends. HfGen uses validation/bot measurements; inspect solvable, botRate, botPieces and score in saved level data and manually test the result.

Worked extension: back up the bank, generate five candidates after the original 25 at moderate difficulty, validate them, then Save. Reopen the JSON and confirm the count changed. An unchecked replace option inserts rather than silently destroying later data. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/HexFit/Resources/hexfit/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "Goal": 0,
  "HasQueue": true,
  "Bees": 0,
  "Flowers": 0,
  "Wax": 0,
  "Voids": 0,
  "GoalTarget": 2,
  "name": "Clover Patch",
  "shape": "hive",
  "hint": "Drag a comb from the tray onto the honeycomb. Fill a whole line to harvest it!",
  "radius": 3,
  "grid": [
    "---....",
    "--.....",
    "-......",
    ".......",
    "......-",
    "p.pp.--",
    "p..p---"
  ],
  "goal": "lines",
  "target": 2,
  "budget": 14,
  "queue": "2.0 3I.0 1.0",
  "seed": 101,
  "difficulty": 0,
  "breather": false,
  "botRate": 1,
  "botPieces": 3,
  "score": 6.4,
  "solved": true
}
```


Root keys: levels. Inspect HfData.cs, HfGen.cs, HfLogic.cs, HfSession.cs, HfLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. For Word Hunt, words, masks and placements are gameplay data; interface translation is a separate task.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/HfArt.cs`. Find `Honey`, currently `#F2B33D`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`HexFitGame.cs:179 / void StartEndless()`	`Honey, HfArt.Get("win_badge"), null, ("Start", Good, (Action)Begin));`
`HexFitGame.cs:245 / void ShowIntro()`	`HfPopup("Meadow Mode open!", "The endless meadow is open.\nTap MEADOW in the top bar any time\nto chase your best score.", Honey, HfArt.Get("win_badge"), null, ("Lovely!", Good, (Action)ShowIntro));`
`HexFitGame.cs:265 / void ShowPreLevel()`	`var p = HfPopup("Level " + Level, msg, Honey, PreLevelArt(), D.name + (D.breather ? " - breather" : ""), ("Play", Good, (Action)Begin));`
`HexFitGame.cs:385 / void GameOver(string why)`	`buttons.Add((outOfCombs ? $"+{HfRules.RescuePieces} combs (watch ad)" : "Puff some smoke (watch ad)", HoneyDeep, () =>`
`HexFitGame.cs:389 / void GameOver(string why)`	`buttons.Add(("Retry Level", Honey, StartLevel));`
`HexFitGame.cs:415 / void EndlessOver()`	`buttons.Add(("Puff some smoke (watch ad)", HoneyDeep, () => HzPlatform.Rewarded(GameId, "rescue_endless", ok => { if (ok) Rescue(false); }));`
`HexFitGame.cs:419 / void EndlessOver()`	`buttons.Add(("Back to Levels", Honey, StartLevel));`
`HexFitGame.cs:755 / void OfferCharge(int b)`	`("Watch ad", HoneyDeep, () => HzPlatform.Rewarded(GameId, "booster_" + BoostShort[b].ToLowerInvariant(), ok =>`
`HexFitGame.Hud.cs:77 / void BuildHudPortrait(float W, float H)`	`var pill = CardAt("LevelPill", 40 + 230, SafeTop + 66, 460, 128, HoneyDeep, 44);`

`HexFitGame.Hud.cs:119 / void BuildHudLandscape(float W, float H)`	`var lvl = CardAt("LevelPill", px, 172, pw, 104, HoneyDeep, 40);`
`HexFitGame.Hud.cs:195 / void ComboLabel(Transform card)`	`_comboTxt = HzUI.Label("Combo", card, "", 28, HoneyDeep, TextAnchor.LowerCenter, false); _comboTxt.fontStyle = FontStyle.Bold;`
`HexFitGame.Hud.cs:217 / void GoalCard(Image card, float w, float h)`	`_goalIconTxt = HzIcons.Icon(card.transform, HzIcons.Hexagon, 58, HoneyDeep);`
`HexFitGame.Hud.cs:277 / void ConfirmLeave(string title, string msg, Action go) => HfPopup(title, msg, Honey, null, null, ("Yes", Good, go), ("Stay", Locked, null));`	`void ConfirmLeave(string title, string msg, Action go) => HfPopup(title, msg, Honey, null, null, ("Yes", Good, go), ("Stay", Locked, null));`
`HexFitGame.Hud.cs:497 / void RefreshBoosters()`	`// Honey popups (HzPopup component so the kit's close and PopupOpen checks work)`
`HexFitGame.Hud.cs:623 / HzPopup HfPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`var st = HzUI.Label("Sub", card.transform, subtitle, 40, HoneyDeep); st.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(cw - 80, 50), new Vector2(0, y - 25));`
`HexFitGame.Hud.cs:725 / void ShowSettingsPopup()`	`("How to Play", Honey, () => HelpPage(0)),`
`HexFitGame.Hud.cs:727 / void ShowSettingsPopup()`	`(_endless ? "New Run" : "Restart Level", HoneyDeep, () => { if (_endless) StartEndless(); else StartLevel(); }),`
`HexFitGame.Hud.cs:843 / void StartTutorial()`	`var ring = HzUI.Img("TutRing", _fxLayer, HfArt.Ring, HoneyDeep); ring.raycastTarget = false; ring.preserveAspect = false;`
`HexFitGame.Hud.cs:903 / void AnimateTutorialTarget()`	`for (int k = 0; k < _tutRings.Count; k++) if (_tutRings[k]) _tutRings[k].color = new Color(HoneyDeep.r, HoneyDeep.g, HoneyDeep.b, a);`
`HexFitGame.View.cs:347 / void BuildView()`	`var rim = HzUI.Img("Rim", bg.transform, HzUI.RoundRect(40, 4), new Color(Honey.r, Honey.g, Honey.b, 0.7f)); rim.type = Image.Type.Sliced; rim.preserveAspect = false; rim.raycastTarget = false; rim.rectTransform.Stretch(2);`

HexFitGame.View.cs:813 / void AnimatePlacement(MoveResult res, byte color)	if (res.Streak >= 2) { HzAudio.PlayCombo(Sfx.Combo, res.Streak - 2, 0.6f, 2f); FloatText("Buzz x" + res.Streak, at + new Vector2(0, -_hs * 1.6f), HoneyDeep, (int)(_hs * 0.9f), 1.0f); }
HfGen.cs:139 / public static bool Play(HfLevelData d, float noise, int noiseSeed, out int used, out int score, out int lines, int budgetOverride = 0)	"Wildflower Way", "Honeysuckle Hedge", "Mint Marsh", "Cowslip Corner", "Sunflower Field", "Bluebell Wood", "Marigold Mile", "Primrose Path",
HfLogic.cs:343 / public string Dump()	public static readonly string[] Praise = { "", "", "Sweet!", "Buzzing!", "Honey-licious!", "Queen Bee!", "Royal Jelly!" };

Source	Exact current code
HexFitGame.cs:65 / class/static declarations	public static readonly string[] BoostColor = { "#E8962A", "#8C7A6B", "#3FBF6F", "#4FA3FF" };
HfArt.cs:9 / class/static declarations	public static readonly Color Ink = HzUI.Hex("#4A2A12"), Honey = HzUI.Hex("#F2B33D"), HoneyDeep = HzUI.Hex("#D98A1C"), Wax = HzUI.Hex("#FBE7B5"),
HfArt.cs:10 / class/static declarations	WaxDim = HzUI.Hex("#E9C98A"), Frame = HzUI.Hex("#8A5222"), FrameDark = HzUI.Hex("#5E3416"), Cream = HzUI.Hex("#FFF6E0"),
HfArt.cs:11 / class/static declarations	Card = HzUI.Hex("#FFF3D6"), CardText = HzUI.Hex("#4A2A12"), Sub = HzUI.Hex("#9A6A3A"), Gold = HzUI.Hex("#FFD23C"),
HfArt.cs:12 / class/static declarations	Good = HzUI.Hex("#3FBF6F"), Bad = HzUI.Hex("#E8455E"), Sky = HzUI.Hex("#4FA3FF"), Lavender = HzUI.Hex("#9B7BFF"), Leaf = HzUI.Hex("#5DB84A"),
HfArt.cs:13 / class/static declarations	Locked = HzUI.Hex("#A89C8A"), BgTop = HzUI.Hex("#8FD3FF"), BgBot = HzUI.Hex("#7CCB5B"), Pink = HzUI.Hex("#FF6FA8");
HfArt.cs:18 / class/static declarations	Color.white, HzUI.Hex("#FF6FA8"), HzUI.Hex("#9B7BFF"), HzUI.Hex("#FFC92E"), HzUI.Hex("#3FCF95"), HzUI.Hex("#FF8A3D"), HzUI.Hex("#4FA3FF"), HzUI.Hex("#E8455E"),

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/HexFitGame.Hud.cs`, find the displayed literal `"Hex Fit"`, and replace its displayed value with `"Meadow Fit"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. BoostName, BoostShort, BoostUnlock and BoostStart in HexFitGame.cs; BoostDesc and guide strings in HexFitGame.Hud.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`HexFitGame.cs:57 / class/static declarations`	`public static readonly string[] BoostName = { "Spin", "Smoker", "Fresh Combs", "Drip" };`
`HexFitGame.cs:177 / void StartEndless()`	`HfPopup("Meadow Mode", "No goal, no limit: harvest the\nhive for as long as combs fit.\nBest score: " + Best.ToString(CultureInfo.InvariantCulture),`
`HexFitGame.cs:245 / void ShowIntro()`	`HfPopup("Meadow Mode open!", "The endless meadow is open.\nTap MEADOW in the top bar any time\nto chase your best score.", Honey, HfArt.Get("win_badge"), null, ("Lovely!", Good, (Action)ShowIntro));`
`HexFitGame.cs:265 / void ShowPreLevel()`	`var p = HfPopup("Level " + Level, msg, Honey, PreLevelArt(), D.name + (D.breather ? " - breather" : ""), ("Play", Good, (Action)Begin));`
`HexFitGame.cs:391 / void GameOver(string why)`	`HzTween.Delay(this, 0.7f, () => HfPopup(outOfCombs ? "Out of Combs!" : "No Room Left!", msg, Bad, null, D.name, buttons.ToArray()));`
`HexFitGame.cs:421 / void EndlessOver()`	`HzTween.Delay(this, 0.7f, () => HfPopup(record ? "New Best!" : "No Room Left!", msg, record ? Good : Bad, record ? HfArt.Get("win_badge") : null, "Meadow Mode", buttons.ToArray()));`

HexFitGame.cs:439 / void Rescue(bool moreCombs)	Toast($"+{HfRules.RescuePieces} combs!", Gold);
HexFitGame.cs:457 / void Rescue(bool moreCombs)	Toast("The smoke cleared some combs!", Gold);
HexFitGame.cs:669 / void OnBooster(int b)	Toast(BoostName[b] + " unlocks at level " + BoostUnlock[b]); HzAudio.Play(Sfx.Invalid, 0.4f);
HexFitGame.cs:699 / void OnBooster(int b)	else Toast(b == BSpin ? "Tap a tray comb to spin it" : b == BDrip ? "Tap a tray comb to melt it into one drop" : "Tap the hive: smoke clears a little flower of 7 cells", null, 1.4f);
HexFitGame.cs:753 / void OfferCharge(int b)	HfPopup("+1 " + BoostName[b], "Out of " + BoostName[b] + ".\nWatch a short ad for one more?", HzUI.Hex(BoostColor[b]), HfArt.Get(BoostArt[b]), null,
HexFitGame.cs:779 / void UseSpin(int t)	if (S.Tray[t].Size == 1 || Shapes.Variants(S.Tray[t].Base).Count == 1) { Toast("That comb looks the same every way", null, 1.2f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }
HexFitGame.cs:811 / void UseDrip(int t)	if (S.Tray[t].Size == 1) { Toast("That comb is already a single drop", null, 1.2f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }
HexFitGame.cs:861 / void UseSmoke(int a)	if (!any) { Toast("Aim at some combs", null, 1.1f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }
HexFitGame.cs:1069 / string PopupTitle()	return t ? t.GetComponent<Text>().text.Replace(' ', '_') : "popup";
HexFitGame.Hud.cs:79 / void BuildHudPortrait(float W, float H)	_levelTxt = HzUI.Label("Level", pill.transform, "", 54, Color.white); _levelTxt.AddOutline(new Color(0.3f, 0.15f, 0, 0.45f), 3);
HexFitGame.Hud.cs:83 / void BuildHudPortrait(float W, float H)	_nameTxt = HzUI.Label("Name", pill.transform, "", 30, new Color(1, 1, 1, 0.95f), TextAnchor.MiddleCenter, false); _nameTxt.fontStyle = FontStyle.Bold;
HexFitGame.Hud.cs:95 / void BuildHudPortrait(float W, float H)	_budgetCard = CardAt("Budget", W / 2, y, cw, ch, Card); _budgetLbl = CardLabel(_budgetCard, "COMBS", out _budgetTxt, 72);

`HexFitGame.Hud.cs:97 / void BuildHudPortrait(float W, float H)`	`var sc = CardAt("Score", W - 40 - cw / 2, y, cw, ch, Card); CardLabel(sc, "SCORE", out _scoreTxt, 62);`
`HexFitGame.Hud.cs:113 / void BuildHudLandscape(float W, float H)`	`var title = HzUI.Label("Title", _layer, "Hex Fit", 78, Gold, TextAnchor.MiddleLeft);`
`HexFitGame.Hud.cs:121 / void BuildHudLandscape(float W, float H)`	`_levelTxt = HzUI.Label("Level", lvl.transform, "", 46, Color.white, TextAnchor.MiddleCenter);`
`HexFitGame.Hud.cs:127 / void BuildHudLandscape(float W, float H)`	`_nameTxt = HzUI.Label("Name", lvl.transform, "", 30, new Color(1, 1, 1, 0.95f), TextAnchor.MiddleCenter, false); _nameTxt.fontStyle = FontStyle.Bold;`
`HexFitGame.Hud.cs:139 / void BuildHudLandscape(float W, float H)`	`_budgetCard = CardAt("Budget", px + cw / 2 + 10, 460, cw, 200, Card); _budgetLbl = CardLabel(_budgetCard, "COMBS", out _budgetTxt, 80);`
`HexFitGame.Hud.cs:141 / void BuildHudLandscape(float W, float H)`	`var sc = CardAt("Score", px, 655, pw, 160, Card); CardLabel(sc, "SCORE", out _scoreTxt, 62);`
`HexFitGame.Hud.cs:175 / void BuildHudLandscape(float W, float H)`	`var t = HzUI.Label("Title", card.transform, title, 30, Sub, TextAnchor.UpperCenter, false);`
`HexFitGame.Hud.cs:181 / void BuildHudLandscape(float W, float H)`	`value = HzUI.Label("Value", card.transform, "", valueSize, CardText);`
`HexFitGame.Hud.cs:195 / void ComboLabel(Transform card)`	`_comboTxt = HzUI.Label("Combo", card, "", 28, HoneyDeep, TextAnchor.LowerCenter, false); _comboTxt.fontStyle = FontStyle.Bold;`
`HexFitGame.Hud.cs:209 / void GoalCard(Image card, float w, float h)`	`_goalLbl = HzUI.Label("Title", card.transform, "GOAL", 30, Sub, TextAnchor.UpperCenter, false);`
`HexFitGame.Hud.cs:221 / void GoalCard(Image card, float w, float h)`	`_goalTxt = HzUI.Label("Value", card.transform, "", 58, CardText);`
`HexFitGame.Hud.cs:245 / void IconRow(float rightX, float y, float size)`	`var eb = HzUI.Button("Meadow", _layer, _endless ? "LEVELS" : "MEADOW", open ? Leaf : Locked, Color.white, OnEndlessButton, 38, (int)(size * 0.3f));`

HexFitGame.Hud.cs:267 / void OnEndlessButton()	if (Level < EndlessUnlock) { Toast("Meadow Mode opens at level " + EndlessUnlock); HzAudio.Play(Sfx.Invalid, 0.4f); if (_endlessBtn) HzTween.Shake(_endlessBtn, 10, 0.3f); return; }
HexFitGame.Hud.cs:277 / void ConfirmLeave(string title, string msg, Action go) => HfPopup(title, msg, Honey, null, null, ("Yes", Good, go), ("Stay", Locked, null));	void ConfirmLeave(string title, string msg, Action go) => HfPopup(title, msg, Honey, null, null, ("Yes", Good, go), ("Stay", Locked, null));
HexFitGame.Hud.cs:313 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)	var btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(bb), 40, (int)(size * 0.32f));
HexFitGame.Hud.cs:357 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)	v.BadgeText = HzUI.Label("N", bd.transform, "", (int)(size * 0.24f), Color.white); v.BadgeText.rectTransform.Stretch();
HexFitGame.Hud.cs:361 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)	v.Caption = HzUI.Label("Cap", v.Root, "", 30, Cream);
HexFitGame.Hud.cs:379 / void RefreshHud()	_levelTxt.text = _endless ? "MEADOW MODE" : "LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture);
HexFitGame.Hud.cs:381 / void RefreshHud()	_nameTxt.text = _endless ? "Best " + Best.ToString(CultureInfo.InvariantCulture) : D.name + (D.breather ? " - breather" : "");
HexFitGame.Hud.cs:385 / void RefreshHud()	_comboTxt.text = S.Streak >= 2 && S.ComboLeft > 0 ? "Buzz x" + S.Streak : "";
HexFitGame.Hud.cs:391 / void RefreshHud()	_goalLbl.text = "BEST"; _goalTxt.text = Mathf.Max(Best, S.Score).ToString(CultureInfo.InvariantCulture); _goalTxt.color = CardText; _goalTxt.fontSize = 46;
HexFitGame.Hud.cs:393 / void RefreshHud()	_budgetLbl.text = "LINES"; _budgetTxt.text = S.Lines.ToString(CultureInfo.InvariantCulture); _budgetTxt.color = CardText;
HexFitGame.Hud.cs:403 / void RefreshHud()	_goalLbl.text = S.Goal == GoalKind.Score ? "TARGET" : D.GoalText(true);
HexFitGame.Hud.cs:407 / void RefreshHud()	_goalTxt.text = S.Goal == GoalKind.Score ? prog + "\n/" + S.Target : prog + "/" + S.Target;

`HexFitGame.Hud.cs:413 / void RefreshHud()`	`_budgetLbl.text = "COMBS LEFT";`
`HexFitGame.Hud.cs:475 / void RefreshBoosters()`	`v.BadgeText.text = c > 0 ? c.ToString(CultureInfo.InvariantCulture) : "+";`
`HexFitGame.Hud.cs:479 / void RefreshBoosters()`	`v.Caption.text = un ? BoostShort[b] : "Lv " + BoostUnlock[b];`
`HexFitGame.Hud.cs:613 / HzPopup HfPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 70, Color.white); t.rectTransform.Stretch(6);`
`HexFitGame.Hud.cs:623 / HzPopup HfPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`var st = HzUI.Label("Sub", card.transform, subtitle, 40, HoneyDeep); st.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(cw - 80, 50), new Vector2(0, y - 25));`
`HexFitGame.Hud.cs:633 / HzPopup HfPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 42, Ink, TextAnchor.MiddleCenter, false);`
`HexFitGame.Hud.cs:703 / void UnlockPopup(int b, Action done)`	`HfPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), HfArt.Get(BoostArt[b]), null, ("Great!", Good, done));`
`HexFitGame.Hud.cs:719 / void ShowSettingsPopup()`	`var p = HfPopup("Settings", "", Frame, null, null,`
`HexFitGame.Hud.cs:725 / void ShowSettingsPopup()`	`("How to Play", Honey, () => HelpPage(0)),`
`HexFitGame.Hud.cs:747 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`HexFitGame.Hud.cs:749 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`HexFitGame.Hud.cs:777 / void HelpPage(int i)`	`HfPopup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], Sky, i == 0 ? HfArt.Get("mascot") : null, null,`

`HexFitGame.Hud.cs:779 / void HelpPage(int i)`	`(last ? "Got it" : "Next", Good, last ? (Action)null : () => HelpPage(i + 1)));`
`HexFitGame.Hud.cs:807 / void ShowTip(string text)`	`var tt = HzUI.Label("T", tip.transform, text, IsPortrait ? 32 : 36, Ink, TextAnchor.MiddleCenter, false);`
`HexFitGame.Hud.cs:1051 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", Gold, 1.3f);`
`HexFitGame.View.cs:1025 / void FloatText(string text, Vector2 at, Color color, int size, float dur)`	`var l = HzUI.Label("Float", _fxLayer, text, Mathf.Max(30, size), color);`
`HexFitGame.View.cs:1045 / void Praise(string text, bool big = false)`	`var l = HzUI.Label("Praise", _fxLayer, text, big ? 104 : 90, Gold);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/hexfit through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Music is Resources/hexfit/Music/meadow_loop.ogg. Preserve its load key when replacing it, check loop boundaries/compression and test Sound and Music toggles separately. Kit UI uses supplied fonts; validate glyph coverage for new languages and retain the actual font licence files.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed puzzle, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.hexfit`, version, scene orientation and signing. The build command writes `Builds/Android/hexfit.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/hexfit/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.